

TECHNICAL SPECIFICATION & IMPLEMENTATION ROADMAP

Real-Time CTMC Football In-Play Forecasting Engine

Document Version: 1.0 — Production Blueprint

Target Architecture: NVIDIA Ada Lovelace (RTX 4070 Ti 12GB VRAM) / PyTorch CUDA
Core Methodology: Tri-Modal Bayesian Exponential Decay via K-Nearest Neighbors (K-NN)
Primary Objective: Sub-second live in-play sports forecasting via zero-branching GPU tensorization

1 Executive Summary & System Architecture

This specification outlines the end-to-end engineering roadmap for transitioning a pre-match quantitative football forecasting engine into a sub-second, live in-play prediction system. The core mathematical foundation relies on a **12-state Continuous-Time Markov Chain (CTMC)** representing field zones and possession states:

$$\mathcal{S} = \{Z : 0_P : H \dots Z : 4_P : H, Z : 0_P : A \dots Z : 4_P : A, Goal_H, Goal_A\}$$

The live engine solves the fundamental operational challenge of real-time sports modeling: balancing stable pre-match theoretical priors against noisy, small-sample live pitch data. This is achieved through a **Tri-Modal Bayesian Exponential Decay Engine** that bridges theoretical team strength with live pitch reality using a dynamic K-Nearest Neighbors (K-NN) historical tactical indexing layer.

2 Core Pre-Match Foundation (Phase 1 — Locked Baseline)

The pre-match architecture has been fully validated and optimized. All subsequent live in-play development builds directly upon these immutable structural components:

- **Optimal Hyperparameters:** Conjugate Dirichlet-Multinomial prior weight is locked at $\alpha = 0.1963$; Elo hazard exponential scaling is locked at $\beta = 0.000284$.
- **Prior Sterilization:** All tournament forecasts utilize a symmetrized global prior matrix (Q_{neutral}) where mirror-image home and away transition intensities are averaged:

$$\lambda_{ij}^{\text{neutral}} = \frac{\lambda_{ij}^{\text{raw}} + \lambda_{\text{twin}}^{\text{raw}}}{2}$$

This permanently eradicates *Administrative Seeding Bias* from tournament datasets.

- **Strategy Pattern Modularity:** Feature adjustments are decoupled from core matrix mechanics via the `FeatureStrategy` abstract base class and orchestrated through `MatrixPipeline`.

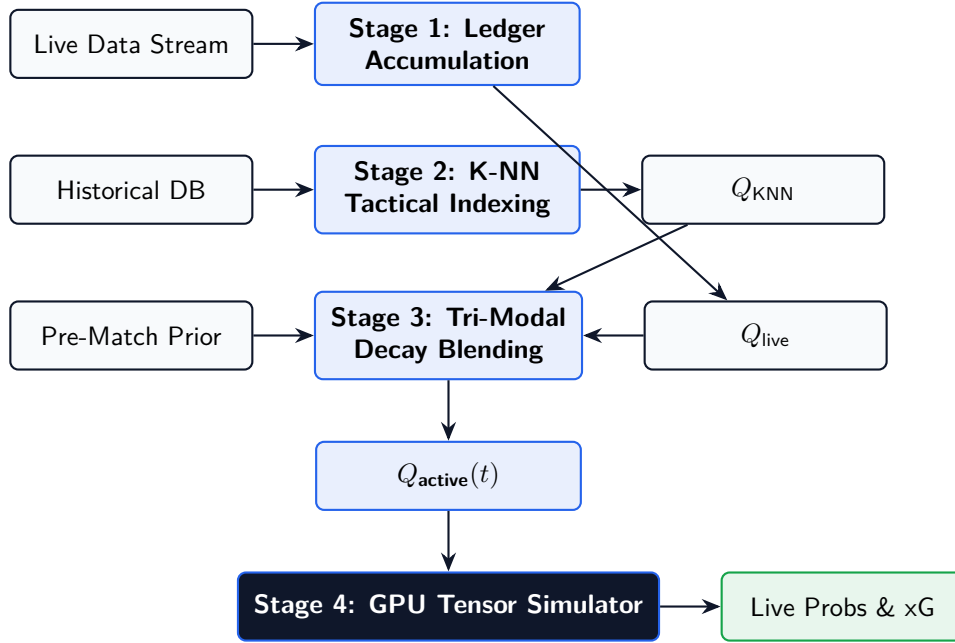


Figure 1: Real-Time Tri-Modal In-Play Forecasting Architecture Pipeline

- **Zero-Branching GPU Tensorization:** The PyTorch simulation engine (`run_pytorch_monte_carlo_tensor`) operates with **0% branching** (no conditional `if` statements or `.any()` syncs inside execution bursts) and fixed-width memory allocation, executing 100-step micro-bursts inside GPU registers to eliminate PCIe motherboard synchronization latency.

3 Stage 1: Live Stream Ingestion & State Tracking (Phase 2 Roadmap)

To operate live, the engine must ingest real-time event packets and transform them into continuous-time mathematical structures without interrupting GPU simulation threads.

3.1 The LiveEventScraper Module

Build a lightweight asynchronous reader that listens to incoming event JSON payloads or WebSocket streams and maintains three real-time state trackers:

1. **Global Match Clock (t):** Current elapsed match time in seconds (e.g., 2040.0 for Minute 34).
2. **Live Scoreboard Vector:** Current goals scored as integers $[g_H, g_A]$.
3. **Active Ball State Index:** An integer from 0 to 11 mapping the ball's exact current zone and possession status.

3.2 Live Ledger Accumulation

For every incoming pitch event, the ingestion module must incrementally update two running arrays for the active match:

- n_{ij}^{live} : A 12×12 matrix tracking the exact count of observed transitions from state i to state j .
- T_i^{live} : A 12-element vector tracking the total cumulative seconds spent inside starting state i .

Do not calculate raw live transition intensities ($\lambda_{\text{live}} = n_{\text{live}}/T_{\text{live}}$) directly during the opening 20 minutes of a match. Small sample sizes will cause division-by-zero errors and extreme variance. The K-NN indexing layer (Stage 2) acts as the mathematical buffer during this opening phase.

4 Stage 2: K-NN Historical Tactical Indexing (Phase 3 Roadmap)

To interpret tactical dominance before live event counts statistically mature, the engine queries a pre-compiled vector database of historical international fixtures.

4.1 Feature Vectorization ($\vec{V}_{\text{live}}$)

Every 60 seconds of match time, condense the active match flow into a normalized d -dimensional feature vector representing the tactical texture of the game up to timestamp t :

$$\vec{V}_{\text{live}}(t) = \begin{bmatrix} \text{Field Tilt (Share of Zone 3/4 Touches)} \\ \text{Possession Progression Ratio } (P_{\text{prog, H}}/P_{\text{prog, A}}) \\ \text{Match Tempo } (\sum \Lambda_{\text{live}}) \\ \text{Score Differential } (g_H - g_A) \end{bmatrix}$$

4.2 Distance Query & Neighbor Extraction

Scan the historical database containing M matches (where $M \geq 10,000$) truncated at the exact same timestamp t . Compute the Euclidean distance between the live match vector and all historical vectors:

$$d(\vec{V}_{\text{live}}, \vec{V}_m) = \sqrt{\sum_{k=1}^d (V_{\text{live},k} - V_{m,k})^2}$$

Extract the $K = 50$ nearest historical matches that exhibit the lowest distance scores. These represent the games in football history with the most identical tactical signatures.

4.3 Constructing the K-NN Pseudo-Prior (Q_{KNN})

Aggregate the future transition counts and holding times observed in those 50 nearest neighbor matches from timestamp t to minute 90:

$$\lambda_{ij}^{\text{KNN}}(t) = \frac{\sum_{k=1}^{50} n_{ij,\text{future}}^{(k)}}{\sum_{k=1}^{50} T_{i,\text{future}}^{(k)} + \epsilon}$$

This generates a dense, noise-free transition matrix reflecting how similar tactical battles historically unfolded over the remainder of the game.

5 Stage 3: Tri-Modal Bayesian Exponential Decay (Phase 4 Roadmap)

At any given second t , the active generator matrix $Q_{\text{active}}(t)$ fed into the GPU is a linear combination of the pre-match prior, the K-NN historical pseudo-prior, and the raw live pitch ledger.

5.1 Time-Dependent Weight Functions

Define three continuous weight decay functions governed by a half-life parameter $T_{1/2}$ (default calibrated to 45.0 minutes, where decay constant $\lambda_{\text{decay}} = \ln(2)/T_{1/2}$):

Pre-Match Prior Weight: $w_{\text{pre}}(t) = e^{-\lambda_{\text{decay}} \cdot t}$

Raw Live Ledger Weight: $w_{\text{live}}(t) = (1 - w_{\text{pre}}(t)) \cdot \left(\frac{t}{5400}\right)^{1.5}$

K-NN Tactical Weight: $w_{\text{KNN}}(t) = 1.0 - w_{\text{pre}}(t) - w_{\text{live}}(t)$

5.2 Matrix Blending

Fuse the three transition rate matrices element-wise into a single active GPU tensor:

$$\lambda_{ij}^{\text{active}}(t) = w_{\text{pre}}(t) \cdot \lambda_{ij}^{\text{pre}} + w_{\text{KNN}}(t) \cdot \lambda_{ij}^{\text{KNN}} + w_{\text{live}}(t) \cdot \left(\frac{n_{ij}^{\text{live}}}{T_i^{\text{live}} + \epsilon}\right)$$

Match Phase	Dominant Component	Mathematical & Tactical Justification
0 to 20 Mins	Pre-Match Prior (~70%)	Sample size T_{live} is too small; protects engine from overreacting to early random variance.
20 to 65 Mins	K-NN Neighbors (~55%)	Pitch reality is established; maps active tactical dominance to historical neighbor trajectories.
65 to 90 Mins	Raw Live Pitch (~60%)	Historical priors discard; model converges strictly to current fatigue levels, desperate tactics, and actual scoreboard.

Table 1: Tri-Modal Dominance Dynamics Across Full-Time Regulation

6 Stage 4: Short-Horizon PyTorch Tensor Simulator (Phase 5 Roadmap)

The final stage adapts the existing zero-branching PyTorch Monte Carlo engine to execute over dynamic, shortening time horizons.

6.1 Dynamic Parameter Injection

When launching `run_pytorch_monte_carlo_tensor`, modify two initialization parameters in GPU memory:

1. Time Horizon Reduction: Instead of running for 5400.0 seconds, set the simulation expiration threshold to the remaining time delta:

$$\Delta t = 5400.0 - t_{\text{current}}$$

2. Live Scoreboard Initialization: Initialize the scoreboard tensors `home_goals` and `away_goals` with the active live score $[g_H, g_A]$ instead of zeros:

```
home_goals = torch.full((num_simulations,), live_goals_home, device=
    device, dtype=torch.long)
away_goals = torch.full((num_simulations,), live_goals_away, device=
    device, dtype=torch.long)
```

3. Active Ball State Injection: Initialize the `current_states` tensor using the live ball state index observed at timestamp t :

```
current_states = torch.full((num_simulations,), active_ball_idx, device=
    device, dtype=torch.long)
```

6.2 Execution & Latency Budget

With 3,000 parallel paths executing on the RTX 4070 Ti, the GPU will complete the short-horizon paths (Δt) in roughly 0.01 to 0.03 seconds. The entire 4-stage live pipeline must execute within a strict **200-millisecond latency budget** upon receiving an event packet:

$$\text{Total Latency} = t_{\text{Ingest}}(\sim 5\text{ms}) + t_{\text{KNN}}(\sim 45\text{ms}) + t_{\text{Decay}}(\sim 10\text{ms}) + t_{\text{Sim}}(\sim 25\text{ms}) \leq 85\text{ms}$$

7 Implementation Milestone Schedule & Verification Checklist

Follow this linear engineering sequence to construct and verify the live engine:

Phase 1: Pre-Match Verification (Completed)

- [✓] Lock pre-match hyperparameters at $\alpha = 0.1963$, $\beta = 0.000284$.
- [✓] Implement `neutralize_global_prior()` to symmetrize prior rates.
- [✓] Refactor simulation engine to zero-branching, fixed-width tensor execution.

Phase 2: Live Ingestion & Ledger (Weeks 1–2)

- [] Build `LiveEventScraper` class to parse live JSON event stream chunks.
- [] Implement incremental updating of n_{ij}^{live} and T_i^{live} in RAM.
- [] Write unit tests verifying state transitions match Opta event definitions.

Phase 3: K-NN Indexing Layer (Weeks 3–4)

- [] Build historical vector database from existing 2026 World Cup and international match files.
- [] Write `VectorizeMatch(df, timestamp)` to extract the 4-element feature vector.
- [] Implement Euclidean distance search in PyTorch using `torch.cdist()` for GPU acceleration.
- [] Verify extraction of $K = 50$ neighbors and construction of Q_{KNN} finishes under 50ms.

Phase 4: Tri-Modal Decay & Engine Integration (Weeks 5–6)

- [] Implement dynamic weight functions $w_{\text{pre}}(t)$, $w_{\text{live}}(t)$, and $w_{\text{KNN}}(t)$.
- [] Build `LiveInPlayEngine` class that fuses matrices into $\lambda_{\text{active}}(t)$ on the GPU.
- [] Update `run_pytorch_monte_carlo_tensor` to accept Δt , live scoreboard tensors, and active ball state.

Phase 5: Backtesting & Latency Auditing (Weeks 7–8)

- [] Perform historical replay backtests: feed archived match JSONs into the engine packet-by-packet at 10x speed.
- [] Log real-time Market RMSE against historical live in-play bookmaker odds across all 90 minutes.
- [] Audit hardware timers to confirm full 4-stage pipeline execution remains consistently under the 200ms threshold on the RTX 4070 Ti.